GLOBAL QUEST
TECHNOLOGIES

An ISO 9001 & ISO 21001
Certified Organizaton

The Quest
for your Dream Job
Ends Here!!

www.gqtech.in

Document Question Answering System using RAG (LangChain + Streamlit)

1. Introduction

Retrieval-Augmented Generation (RAG) is an advanced technique used in modern Natural Language Processing systems that combines information retrieval with large language models to produce accurate and context-aware responses. Traditional language models generate answers based only on their training data, whereas RAG systems retrieve relevant information from external documents before generating responses.

In this project, a Document Question Answering System was developed using the RAG approach. The system allows users to upload a PDF document and ask questions related to the content of that document. The system processes the document, converts the text into vector embeddings, stores them in a vector database, and retrieves the most relevant information when a query is asked.

The project uses LangChain for building the RAG pipeline, HuggingFace Embeddings for converting text into vectors, and FAISS as a vector database for efficient similarity search. The system was tested using a document containing information about Paracetamol.

This approach ensures that answers generated by the system are based on the content of the document rather than relying solely on the language model's internal knowledge.

2. System Architecture

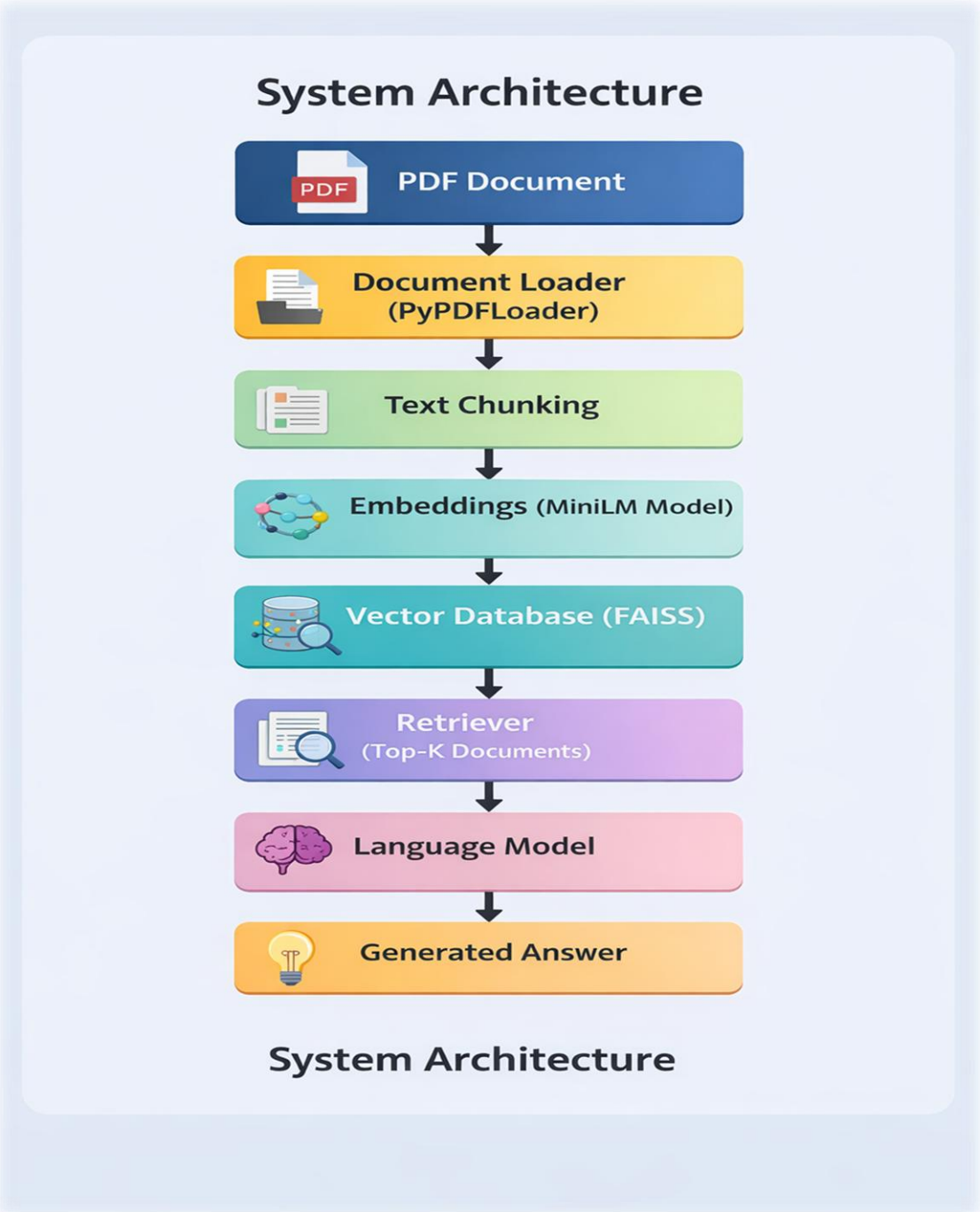
The architecture of the system consists of multiple stages that work together to implement the RAG pipeline.

The process begins when a user uploads a PDF document. The system reads the document and extracts the text using a document loader. Since large documents cannot be processed efficiently in their original form, the text is divided into smaller segments called chunks. These chunks are then converted into embeddings using a sentence transformer model.

The embeddings are stored in a FAISS vector database. When a user asks a question, the query is also converted into an embedding. The system compares

this query embedding with the stored embeddings and retrieves the most relevant chunks from the document. These retrieved chunks are then provided to a language model, which generates the final answer.

System Architecture Flow



3. Document Processing and Chunking Strategy

Documents often contain large amounts of text, which makes it difficult for language models to process them efficiently. To address this issue, the document is divided into smaller segments known as text chunks.

In this project, the Recursive Character Text Splitter method from LangChain was used to divide the document into chunks.

The parameters used were:

ChunkSize=500
Chunk Overlap = 50

Chunk size determines the maximum number of characters present in each chunk. A chunk size of 500 characters ensures that each chunk contains sufficient contextual information while still being small enough to process efficiently.

Chunk overlap is used to maintain continuity between consecutive chunks. An overlap of 50 characters ensures that important information located at the boundary of chunks is not lost. This helps preserve context when retrieving information during query processing.

The chunking strategy improves the efficiency of the retrieval process and ensures that the system retrieves relevant information accurately.

4. Embedding Concept

Embeddings are numerical vector representations of text that capture semantic meaning. Instead of representing words as simple tokens, embeddings convert text into high-dimensional vectors that represent the meaning and context of the text.

In this project, HuggingFaceEmbeddings were used with the model:

sentence-transformers/all-MiniLM-L6-v2

This model is widely used for semantic similarity tasks because it produces high-quality embeddings while being computationally efficient.

Each text chunk generated during the chunking process is converted into a vector embedding using this model. These embeddings allow the system to compare the semantic similarity between different pieces of text.

When a user asks a question, the query is also converted into an embedding vector. The system then compares the query vector with stored document vectors to determine which document chunks are most relevant.

This process enables efficient semantic search and forms the foundation of the retrieval mechanism used in the RAG system.

5. Vector Database and Retrieval Process

To efficiently store and search embeddings, the system uses FAISS (Facebook AI Similarity Search) as the vector database.

FAISS is a library designed for fast similarity search and clustering of dense vectors. It allows the system to quickly identify vectors that are most similar to a given query vector.

In this project, embeddings generated from document chunks are stored in the FAISS vector database. When a user submits a query, the query is converted into an embedding vector using the same embedding model.

The FAISS database then performs a similarity search between the query embedding and the stored document embeddings. The system retrieves the Top-3 most similar chunks based on cosine similarity.

These retrieved chunks contain the information that is most relevant to the user's question. The chunks are then passed to the language model, which generates the final answer.

This retrieval step ensures that the language model receives relevant context before generating responses, improving both accuracy and reliability.

6. Question Answering with Retrieval-Augmented Generation

Once the relevant document chunks are retrieved, they are passed to a language model that generates the final answer. In this project, a HuggingFace pipeline using the GPT-2 model was used for text generation.

The language model receives two inputs:

1. The user's query
2. The retrieved document chunks

Using this context, the model generates a response that is grounded in the information retrieved from the document.

This approach ensures that the answers generated by the system are based on the uploaded document rather than relying solely on the language model's internal knowledge.

7. Conclusion

This project demonstrates how Retrieval-Augmented Generation can be used to build intelligent document-based question answering systems. By integrating document retrieval with language models, the system can generate accurate and context-aware responses.

The use of text chunking, embeddings, vector databases, and retrieval pipelines enables efficient processing of large documents and improves the quality of generated answers.

Such systems can be applied in many real-world domains, including healthcare documentation, legal document analysis, research papers, customer support systems, and enterprise knowledge management systems.

The project successfully demonstrates the implementation of a RAG-based question answering system using LangChain, HuggingFace embeddings, and FAISS.

GLOBAL QUEST
TECHNOLOGIES

fuel your
passion for
IT with
our **guidance.**

Global
Quest Technologies

#324,2nd Floor, 3 A Cross, Near
Seshadripuram First Grade College,
Above City Union Bank,
Yelahanka New Town,
Bengaluru-560064

+91 9448 403 469 | 080-49720009
info@gqtech.in | www.gqtech.in